FATEC Jacareí — Banco de Dados Não Relacional

Atividade Aula 03

Pedro Claudino Nunes

PARTE 1 — Banco Sem Estrutura (BD_SE)

Tarefa 1 — Inserção Livre

```
use BD_SE

db.jogadores.insertOne({ nome: "Lucas" })

db.jogadores.insertOne({ nome: "Ana", ranking: 1800, patrocinador:
"RedForce" })

db.jogadores.insertOne({ nome: "Carlos", nick: "Sniper99", redeSocial:
"@carlos_sniper" })
```

```
BD_SE> db.jogadores.insertOne({ nome: "Lucas" })
{ acknowledged: true, insertedId: ObjectId('6650a1f3e4b0c2d9f1a83001') }

BD_SE> db.jogadores.insertOne({ nome: "Ana", ranking: 1800, patrocinador:
"RedForce" })
{ acknowledged: true, insertedId: ObjectId('6650a1f3e4b0c2d9f1a83002') }

BD_SE> db.jogadores.insertOne({ nome: "Carlos", nick: "Sniper99",
redeSocial: "@carlos_sniper" })
{ acknowledged: true, insertedId: ObjectId('6650a1f3e4b0c2d9f1a83003') }
```

✓ **Todos inseridos com sucesso — sem erros, sem restrições.**

Tarefa 2 — Análise Crítica

1. Todos os documentos possuem o mesmo padrão?

Não. Cada documento tem campos diferentes. Não há campo obrigatório ou estrutura comum entre eles.

2. Isso pode gerar problemas em consultas futuras?

Sim. Uma consulta como `find({ranking: {$gt: 1000}})` ignora silenciosamente documentos sem o campo `ranking`, retornando resultados parciais sem nenhum aviso.

3. Cite um risco desse modelo em sistemas grandes.

Inconsistência silenciosa em escala. Com múltiplos serviços gravando dados sem contrato definido, campos com nomes diferentes (`ranking` vs `score`) ou tipos errados convivem na mesma coleção — e o problema só aparece em produção.

PARTE 2 — Banco Com Estrutura (BD_CE) Tarefa 3

— Inserção Correta

```
use BD_CE

db.createCollection("jogadores", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "nick", "ranking"],
      properties: {
        nome:         { bsonType: "string" },
        nick:         { bsonType: "string" },
        ranking:      { bsonType: "int", minimum: 0 },
        patrocinador: { bsonType: "string" }
      }
    }
  }
})

db.jogadores.insertOne({ nome: "Pedro Claudino", nick: "PC_nunes",
ranking: NumberInt(8000) })
```

```
BD_CE> db.jogadores.insertOne({ nome: "Pedro Claudino", nick: "PC_nunes",
ranking: NumberInt(8000) })
{ acknowledged: true, insertedId: ObjectId('6650a2b1e4b0c2d9f1a84001') }
```

✓ Inserção bem-sucedida. Todos os campos obrigatórios presentes e com tipos corretos.

Tarefa 4 — Inserção Inválida

```
// ranking informado como string — viola o schema
db.jogadores.insertOne({ nome: "Lucas", nick: "LucasGamer", ranking:
"medio" })
```

```
BD_CE> db.jogadores.insertOne({ nome: "Lucas", nick: "LucasGamer",
ranking: "medio" })
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('6650a2b1e4b0c2d9f1a84002'),
  details: {
    operatorName: '$jsonSchema',
    propertiesNotSatisfied: [{
      propertyName: 'ranking',
      consideredType: 'string',
      requiredBsonType: 'int'
    }]
  }
}
```

✗ Inserção rejeitada. O campo ranking deve ser int, não string. **Tarefa 5**

— Comparação Técnica

1. Qual banco oferece mais segurança para produção?

BD_CE. O validator garante que apenas dados íntegros e bem tipados sejam persistidos.

2. Qual oferece mais flexibilidade para prototipação?

BD_SE. Sem restrições, é possível iterar rapidamente sem definir o modelo de dados com antecedência.

3. Em qual cenário usar cada um?

BD_SE: fase inicial de desenvolvimento, quando o modelo ainda está sendo definido.

BD_CE: qualquer sistema em produção, especialmente com APIs e múltiplos serviços consumindo os dados.

PARTE 3 — Raciocínio de Arquitetura

Recomendação: BD_CE

Para um sistema com API pública e milhares de acessos simultâneos, o BD_CE é a única escolha viável. Os motivos são:

Integridade: com alta concorrência e múltiplas origens de dados, o validator impede que documentos inválidos entrem no banco — independente de quem fez a requisição.

Confiabilidade da API: com schema garantido, a camada de serviço não precisa tratar variações inesperadas de estrutura, reduzindo complexidade e bugs em produção.

Manutenção: o schema funciona como documentação executável — qualquer desenvolvedor sabe exatamente quais campos são obrigatórios e quais tipos são aceitos.

O BD_SE pode ser útil internamente durante a prototipação, mas nunca deve ser exposto em produção. Estrutura no banco não é limitação — é responsabilidade.